

FabOrchestrator PWA Demo Plan (v2)

Goal

Build a small **FabOrchestrator Progressive Web App demo** for Jothi that is strongly connected to a real manufacturing use case rather than being a generic AI app.

The use case is a **Production Order Exception Assistant**.

The problem statement:

A shop-floor supervisor needs a simple mobile-friendly way to find a production order, understand any problem affecting it, see the relevant MES data, and get a clear recommended next action without digging through multiple screens or raw system information.

The main demo flow:

Search or scan production order → see current order status and problem(s) → AI explains the problem → AI shows verifiable evidence → AI recommends an action → supervisor approves/rejects/escalates → action is recorded.

Scope Assumptions

- **The FabOrchestrator use case is an assumption, not a confirmed requirement.** Jothi's actual ask is a PWA demo built with Claude Design and Claude Code; the Production Order Exception Assistant is our proposed concretization because Jothi consistently wants demos tied to specific use cases. The architecture (adapter pattern, generic rule engine, screen structure) is deliberately adaptable if he redirects to a different use case.
- **This builds inside the existing FabOrchestrator repository**, not a fresh app. Every stack choice below is a *default pending repository inspection* — if FabOrchestrator already has an equivalent (data fetching, validation, persistence, test framework, Claude/Bedrock gateway), reuse it rather than adding a parallel dependency.
- **Hard deadline: a working Tier 0 by Friday.** Everything past Tier 0 is research latitude, which Jothi explicitly granted.
- This plan supersedes earlier drafts on two points: evidence is cited **by record ID** (not prose values like "42 minutes"), and there is **no LLM confidence score** — severity comes from the deterministic rules.

Main User Experience

The user opens a FabOrchestrator-style PWA on a phone, tablet, or desktop.

They search for a production order such as `PO-10382`, or scan a printed order label with the device camera.

The app shows:

- Production order
- Product
- Current operation
- Machine/resource
- Planned quantity
- Completed quantity
- Due time
- Current status
- Defects
- Downtime
- Any detected production problems (there may be more than one)

Example:

- Machine ASM-04 stopped for 42 minutes
- Defect rate increased from 2% to 9%
- Production is behind schedule

The user clicks **Explain Issue**.

While the analysis runs (typically 2–5 seconds), the UI shows an explicit "Analyzing order data..." state — a skeleton panel or streamed text — so the wait reads as work in progress, never as a frozen screen.

The AI returns a **single unified analysis** covering all detected issues:

- Simple explanation of the overall problem
- Likely root cause, with related issues ranked by priority
- Evidence as structured references to real MES records
- Recommended next action
- Rule-derived severity (not a model-invented confidence number)

Example:

Production is behind schedule mainly because ASM-04 was unavailable for 42 minutes. The elevated defect rate (9% vs. a normal 2%) began shortly after the machine restarted, suggesting both issues share a cause.

Recommended action:

Inspect ASM-04 before continuing production.

The supervisor can then:

- Approve
- Reject
- Escalate

The system records the decision in an activity/audit log.

Core Architectural Principle

The app must **not behave like a generic chatbot**.

Responsibilities are separated like this:

MES provides facts → deterministic rules detect problems and assign severity → AI explains the problems and recommends an action, citing only supplied evidence records → human makes the decision.

The LLM never decides whether a machine actually stopped or whether the defect rate is high. Those come from real data and deterministic logic. The LLM's role is narration and recommendation over pre-verified facts.

Technical Architecture

Frontend

- Next.js
- React
- TypeScript
- Existing FabOrchestrator UI components
- Existing `cmf-theme.css`
- **TanStack Query** for production-order data *if the repo doesn't already have a data-fetching pattern* — reuse the existing approach if one exists

The PWA should visually look like an extension of FabOrchestrator.

Latency handling is a first-class UI concern: the Explain Issue flow gets a dedicated loading state (skeleton or streaming) sized for a 2–5 second Bedrock round trip.

PWA Layer

- Next.js web app manifest (Tier 0 — installability is cheap and demos well)
- **Barcode/QR scanning** via the browser `BarcodeDetector` API with a zxing-wasm fallback (promoted to Tier 1–2 — low effort, high demo impact: scan a printed order label, order appears)
- Service worker, offline caching, push notifications remain stretch goals

Backend/API

Next.js Route Handlers; no separate backend. **Follow the repo's existing API pattern if one exists** — these endpoints describe the contract, not necessarily new conventions:

- `GET /api/orders/{id}`
- `GET /api/orders/{id}/issues`
- `POST /api/orders/{id}/analyze`

- `POST /api/orders/{id}/actions`
- `GET /api/orders/{id}/activity`

The frontend communicates only with these APIs.

MES Integration Layer

TypeScript abstraction:

`MESAdapter`

- `getProductionOrder()`
- `getOperations()`
- `getMachineStatus()`
- `getDefects()`
- `getMaterialStatus()`
- `getEvents()`

First demo: `MockMESAdapter` backed by JSON data. Later: `FabOrchestratorMESAdapter` connected to the real MES/data source.

Every record returned by the adapter carries a stable ID (`EVT-2231`, `DEF-0417`, etc.).

These IDs are the backbone of evidence grounding — the LLM cites them, and validation resolves them. The mock data must include them from day one so the real adapter slots in without changing the grounding layer.

Deterministic Rule Layer

Plain TypeScript. Each rule outputs an issue object with a **type**, a **severity**, and the **IDs of the supporting records**:

- Downtime > 30 minutes → `MACHINE_DOWNTIME`
- Defect rate > configured threshold → `QUALITY_PROBLEM`
- Behind schedule → `PRODUCTION_DELAY`
- Required material unavailable → `MATERIAL_SHORTAGE`

Concrete delay definition (previously vague, now explicit):

```
expected_qty = planned_rate_per_hour × hours_elapsed_on_current_operation
delay_ratio = (expected_qty - completed_qty) / expected_qty
PRODUCTION_DELAY fires when delay_ratio > 0.10 (configurable)
```

Without a formula like this the rule fires constantly or never; thresholds live in a single config object so they are tunable and auditable.

Severity comes from the rules, not the model. Example: downtime 30–60 min = `MEDIUM`, > 60 min = `HIGH`; defect rate 2–3× baseline = `MEDIUM`, > 3× = `HIGH`. The UI displays this rule-

derived severity. There is no LLM-invented confidence score anywhere in the product — self-reported LLM confidence is poorly calibrated and reads as false precision.

Multiple rules can fire simultaneously (the demo scenario intentionally fires three). All detected issues are passed to the AI together.

AI Context Builder

Do not send the LLM a database dump. Build a small structured context:

- Production order summary
- Expected vs. actual output (with the numbers the delay rule used)
- Machine status
- **Evidence records, each with its stable ID** — downtime events, defect records, material events
- All detected issues with their severities
- Baseline/normal values (e.g., normal defect rate)

The context is the **complete universe of citable facts**. The model cannot cite anything outside it.

This exact context object is also snapshotted for the audit trail (see below).

LLM Layer

- Claude via AWS Bedrock (Converse API)
- **Check for an existing AthenaTech/FabOrchestrator Claude gateway first** — if the repo already talks to Claude, reuse that integration rather than wiring a second Bedrock client

The LLM performs only:

1. Explain the detected production problems as one unified, prioritized analysis (never one disconnected explanation per issue)
2. Identify the likely root cause and how the issues relate
3. Cite supporting evidence **by record ID**
4. Recommend a single next action

Structured output via forced tool use

Do not prompt-and-hope for JSON. Define the output schema as a **tool** on the Converse API and force the tool call (`(toolChoice)`). Schema conformance comes nearly for free and the markdown-fence-stripping failure mode disappears. Zod still validates the result on top.

Output schema:

```
json
```

```
{
  "summary": "string",
  "likely_cause": "string",
  "issue_priority": ["MACHINE_DOWNTIME", "QUALITY_PROBLEM", "PRODUCTION_DELAY"]
  "evidence": [
    { "record_id": "EVT-2231", "field": "downtime_minutes", "value": 42 },
    { "record_id": "DEF-0417", "field": "defect_rate", "value": 0.09 }
  ],
  "recommended_action": "string"
}
```

Note what is absent: no free-text evidence sentences to parse, and no `confidence` field.

Demo-day resilience

A live Bedrock call is a single point of failure in front of Jothi. The analyze endpoint keeps a **cached known-good response for the demo order** and falls back to it (visibly labeled or not, per taste) on timeout or error. The demo never dies on a network blip.

Validation and Grounding

Zod validates both MES data structures and LLM output.

Because evidence is structured references rather than prose, grounding validation is a **trivial lookup**, not natural-language parsing:

1. Every `record_id` in `evidence` must exist in the context that was sent.
2. Every `field`/`value` pair must match the actual record.
3. Every issue type in `issue_priority` must be one the rule layer actually detected.

Any violation → the response is rejected and regenerated once; a second failure surfaces a flagged, degraded state rather than unverified claims.

Only when validation passes does the UI show the **"Grounded in MES data"** badge — the badge is earned, not decorative.

Human Review

The AI recommends; it never acts. Buttons:

- Approve
- Reject
- Escalate

This loop is cheap to build and completes the demo narrative, so it ships in **Tier 1** (in-memory log is fine at first). Later, decisions can trigger real FabOrchestrator/MES actions: create maintenance request, hold order, notify operator, start quality investigation.

Audit Trail

Store per analysis:

- Detected issues (with rule versions/thresholds in effect)
- **Full snapshot of the exact context object sent to the LLM**
- Raw LLM output
- Validation result
- Recommendation shown
- User decision + timestamp
- Model ID and version

The input snapshot is what makes old analyses reproducible and explainable after rule thresholds change — and it doubles as free test-case material for Promptfoo.

Example activity feed:

- 11:31 — Production delay detected
- 11:32 — Defect threshold exceeded
- 11:33 — AI analysis generated (grounding: passed)
- 11:35 — Supervisor reviewed recommendation
- 11:36 — Recommendation approved

Persistence: **whatever pattern the repo already uses, if any**; otherwise Prisma + SQLite as the simple default. Don't add Prisma alongside an existing data layer.

Deployment check (do this before Tier 3, not after): SQLite + Next.js route handlers works locally but breaks on serverless/ephemeral infrastructure. Confirm how the AthenaTech AWS deployment runs the app (long-lived container vs. Lambda-style). If ephemeral, swap SQLite for RDS/Postgres or run the app in a container with a volume — a five-minute check now versus a rewrite later.

LLM Evaluation

Use **Promptfoo**, seeded with real context snapshots from the audit trail.

Test cases:

Case 1 — Machine downtime. Correct issue identified; correct machine; cites the right event IDs; recommends inspection/maintenance.

Case 2 — High defect rate. Quality issue identified; cites real defect records; recommends quality investigation.

Case 3 — Material shortage. Shortage identified; recommends checking/replenishing material.

Case 4 — Multi-issue order (the demo scenario). Produces one unified analysis; prioritizes sensibly; does not cherry-pick a single issue or emit disconnected explanations.

Automatic assertions:

- Valid tool-call output (schema)
 - Grounding validation passes (all cited IDs exist and match)
 - Correct issue types in `issue_priority`
 - No hallucinated records — directly measurable now, since evidence is by ID
 - Reasonable recommendation (rubric/LLM-graded)
 - Latency and cost
 - Model-to-model regression across Claude models or prompt versions
-

Development Tooling & Workflow (Required)

This project is built using two required Anthropic tools. They don't change the architecture — they define *how* the work gets done, so each tier below assumes this workflow.

Claude Design — screens and visual prototypes

Use Claude Design to produce the screen designs before implementation:

- **Order search screen** — search field, scan button, recently viewed orders (mobile-first)
- **Order detail screen** — status, quantities, machine, detected issues with severity badges, prominent **Explain Issue** button
- **AI analysis panel** — summary, likely cause, evidence list, "Grounded in MES data" badge, loading/skeleton state. Can be a panel on the detail screen rather than a separate page.
- **Approve / Reject / Escalate action bar**
- **Activity feed** (later tier)

Inputs to give Claude Design — the existing app is the visual source of truth:

- Existing FabOrchestrator screenshots
- `app/cmf-theme.css` and relevant components from the repo
- Existing layout and interaction patterns
- The problem statement and user flow, verbatim

Design guardrails — this must read as industrial enterprise software:

- Clear hierarchy; dense but readable information; tables/cards where appropriate
- Clear warning states for detected issues
- Minimal decorative AI styling — no gradients-and-sparkles treatment, no giant chatbot interface. AI is secondary to the manufacturing workflow.

Design review gate (mandatory before implementation): does it solve the actual production-order problem, look like FabOrchestrator, keep AI secondary, and stay understandable to a supervisor at a glance — with no unnecessary screens? **Do not implement a bad design just because Claude Design generated it.** Iterate in Design (cheap) rather than in code (expensive).

Claude Code — implementation

Use Claude Code for the build: the screens, rule engine, `MESAdapter` + mock, Claude integration, validation schemas, grounding checks, and tests — **inside the existing FabOrchestrator repository**.

First session is inspection, not implementation. Before writing code, Claude Code must:

1. Read the repository structure and any existing `CLAUDE.md`
2. Understand existing components, `cmf-theme.css`, and API/data-fetching patterns
3. Find any existing Bedrock/Claude integration and auth setup
4. Determine the smallest set of files that need to change
5. **Report back:** what infrastructure gets reused, what files change, what new files are genuinely necessary, and whether anything in this plan conflicts with the codebase — *then* implement Tier 0

Ongoing workflow rules:

- Keep a `CLAUDE.md` at the repo root capturing the non-negotiables so every session enforces them: evidence cited by record ID only, no LLM-invented confidence, severity comes from rules, forced tool-call output, thresholds in one config object, adapter contract stable from Tier 0, reuse existing patterns over new dependencies.
- Work tier by tier: one tier = one milestone, tests passing before moving on. **Never start Tier 3/4/5 features before Tier 0 works cleanly.** The tier structure doubles as the Claude Code task breakdown.
- Have Claude Code write the tests for the rule layer and grounding validation as it builds them — deterministic correctness is the whole point of those layers. Use the repo's existing test framework; Vitest/Playwright only if none exists.
- Hand Claude Code the approved Claude Design screens (or their specs) when building each UI surface.
- If implementation reveals a UI problem: back to Claude Design → revise → re-implement. The tools are part of the **development workflow**, not the production runtime.

Docs, if needed during setup: <https://docs.claude.com/en/docs/claude-code/overview>

Development Tiers

(Each tier: screens designed/updated in Claude Design first, then implemented via Claude Code.)

Tier 0 — Bare Minimum / Friday

- FabOrchestrator-style Next.js screen
- Production-order search
- Mock JSON order **with stable record IDs baked in**
- Order details

- One hard-coded production problem
- Installable PWA manifest

Tech: Next.js, React, TypeScript, existing FabOrchestrator CSS/components, JSON.

Tier 1 — Meaningful Demo

- Real deterministic rules with severities and the explicit delay formula
- Explain Issue button with proper loading state
- Claude via Bedrock, **forced tool-call structured output**
- Unified multi-issue explanation + recommended action
- **Approve / Reject / Escalate** (in-memory activity log)
- Cached fallback response for the demo order

Tech added: AWS Bedrock, Claude, TypeScript rule engine.

Tier 2 — Strong Demo

- Evidence-by-ID grounding validation with regenerate-then-flag policy
- Zod validation end to end
- "Grounded in MES data" badge (earned)
- Barcode/QR scanning via `BarcodeDetector`
- Activity feed UI

This demonstrates controlled agentic AI rather than a chatbot.

Tier 3 — Production-Like

- **Deployment check first** (SQLite viability on target infra)
- Prisma + SQLite (or Postgres per the check)
- Persistent audit history with full context snapshots
- Multiple production problem scenarios
- Promptfoo harness seeded from audit snapshots, including the multi-issue case

Tier 4 — Real FabOrchestrator Integration

Replace `MockMESAdapter` with `FabOrchestratorMESAdapter`. Because record IDs and the adapter contract were fixed from Tier 0, the grounding layer, rules, and UI do not change. Read real production orders, operations, machines, defects, events, material status.

Tier 5 — PWA Stretch Features

- Push notifications ("PO-10382 requires supervisor attention")
- Offline caching via Serwist
- Deeper mobile install experience

(Barcode scanning has moved up to Tier 2.)

Proposed Technology Stack

(Defaults — every item defers to an existing equivalent found during repository inspection.)

- **Frontend:** Next.js, React, TypeScript, FabOrchestrator components/CSS; TanStack Query only if no existing data-fetching pattern
 - **PWA:** Next.js manifest, `BarcodeDetector` + zxing-wasm fallback, Serwist later
 - **Backend:** Next.js Route Handlers, following existing API conventions
 - **MES integration:** TypeScript adapter pattern with stable record IDs
 - **Mock data:** JSON
 - **Rules:** Plain TypeScript with a single tunable threshold config
 - **Validation:** Zod + ID-based grounding lookups
 - **LLM:** Claude via Bedrock Converse API with forced tool use; existing AthenaTech Claude gateway if present
 - **Persistence:** Existing repo pattern; else Prisma + SQLite (pending deployment check)
 - **Evaluation:** Promptfoo seeded from audit snapshots
 - **Testing:** Existing repo frameworks; else Vitest + Playwright
 - **Design tooling (required):** Claude Design for all screen prototypes and states
 - **Build tooling (required):** Claude Code for implementation, driven by a repo-root `CLAUDE.md` and the tier milestones
 - **Deployment:** Existing AthenaTech AWS infrastructure
-

What NOT to Build

Guardrails against scope drift — this assignment must not become:

- A generic AI chatbot or a giant chat interface
- A drag-and-drop agent builder or generic AI platform
- A sprawling manufacturing dashboard or replacement MES
- An autonomous factory-control agent (the human always decides)
- A multi-page application collecting unrelated AI features

The demo stays one obvious story: **a production order has a problem, FabOrchestrator detects it, AI explains it, the supervisor decides.** Priority throughout: **specific manufacturing value > impressive-looking generic AI features.**

Most Important Design Decisions

1. Do **not** build a generic AI platform. Build one clear manufacturing workflow: **order → problem detected → evidence → AI explanation → recommendation → human decision.**
2. **Evidence is cited by record ID, never by prose.** This single schema decision makes grounding validation trivial, makes hallucination measurable, and makes the "Grounded in MES data" badge honest. It is the highest-leverage choice in the plan.

3. Severity and issue detection are deterministic; the model narrates and recommends over pre-verified facts, and a human always decides.